Tree-Based Methods

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

→ Capacidad predictiva ↑

→ Interpretabilidad ↓

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Recap

~~iid~~

→ dependencia espacial

- Queremos predecir y en funcion de observables ($\mathbf{x}_i$)

$$y = f(\mathbf{x}_i) + u \tag{1}$$

- donde la estimación de f implica la que mejor generalize (prediga mejor fuera de muestra):

$f(x) = X\beta$

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{2}$$

Recap

- Por ejemplo si L es la perdida cuadrática y $f(x_i) = \mathbf{x}_i\beta$ el problema se resume a encontrar los β.

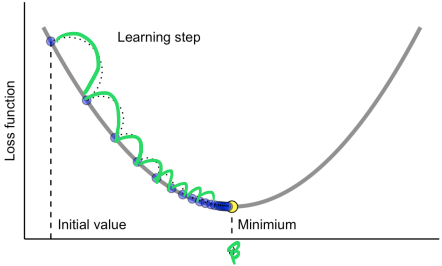
$$\hat{\beta} = \underset{\beta}{\text{argmin}} \left\{ \sum_{i=1}^{n} (y_i - \mathbf{x}_i\beta)^2 \right\} \tag{3}$$

$f(x) = X\beta$

$\rightarrow \hat{\beta} = \underline{(X'X)^{-1}} X'y$

$(X'X)\beta = X'y$

- para estimar los β usamos en ML gradiente descendiente



Source: Boehmke, B., & Greenwell, B. (2019)

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Motivación

- Queremos predecir:

$f(x_i)$

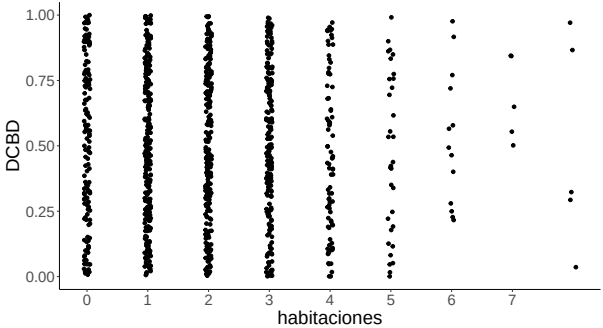
- x_i
- f

LPM
Logit
ED
LDA, QDA
Arboles

$$Precio = f(\text{habitaciones}, \text{ubicación}) \tag{4}$$

- cat pon bus

- Equipos



Árboles

- Podemos asumir f es lineal

$$f(\mathbf{x}_i) = \beta_0 + \beta_1 Habitaciones_i + \beta_2 DCBD_i + u_i \tag{5}$$

Árboles

- Podemos asumir f es lineal

$$f(\mathbf{x}_i) = \beta_0 + \beta_1 Habitaciones_i + \beta_2 DCBD_i + u_i \tag{5}$$

$\beta_2 DCBD + \beta_3 DCBD^2 + \beta_4 DCBD^3 + \ldots$

- o ajustar un modelo flexible e interpretable como son los arboles

$$f(x_i; \{R_j, \gamma_j\}_1^J) = T(x_i; \{R_j, \gamma_j\}_1^J) \tag{6}$$

- donde

$$T(x_i; \{R_j, \gamma_j\}_1^J) = \sum_{j=1}^{J} \gamma_j I(x \in R_j) \tag{7}$$

Árboles

- El problema de optimización

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{8}$$

- es ahora

$$\{\hat{R}_j, \hat{\gamma}_j\}_1^J = \underset{\{R_j, \gamma_j\}_1^J}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, T(\mathbf{x}_i; \{R_j, \gamma_j\}_1^J)) \right\} \tag{9}$$

$L(y, \hat{y})^2 \rightarrow \hat{\gamma} = \frac{\sum \hat{y}}{\#[R_i]}$

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Bat. compv.

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Bagging

- Problema con CART: pocos robustos.
- Idea: podemos mejorar mucho el rendimiento mediante la agregación

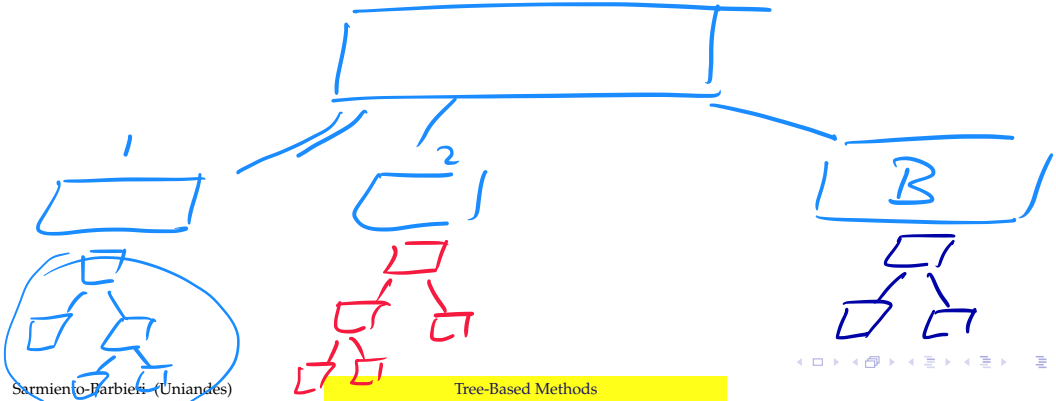
$$x_1, \dots x_n \sim iid\,(\mu, \sigma^2)$$

$$\bar{x} = \frac{\sum x_i}{n} \rightarrow E(\bar{x}) = \frac{\sum E(x_i)}{n} = \frac{n\mu}{n} = \mu$$

$$V(\bar{x}) = V\left(\frac{\sum x_i}{n}\right) = \frac{1}{n^2} V(\sum x_i) = \frac{1}{n^2} \sum V(x_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$$

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).



Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Random Forests

$V(\hat{f}) = \frac{\sigma^2}{B} \downarrow$

dependencia

$Var\left(\sum^{B} T_i\right) \geq \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B}$

- Problema con el bagging: si hay un predictor dominante en la muestra de entrenamiento, diferentes árboles son muy similares entre sí.

? correlación

$\rho = 1 \quad Var = \boxed{\sigma^2}$

- Bosques (forests): reduce la correlación entre los árboles en el boostrap.

$\rho = 0 \quad Var = \frac{\sigma^2}{B} \downarrow$

- Si hay p predictores, en cada partición use solo $m < p$ predictores, elegidos al azar.
- Bagging es forests con $m = p$ (usando todo los predictores en cada partición).
- m es un hiper-parámetro, $m = \sqrt{(p)}$ es un benchmark

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).
  - Para cada muestra, ajuste un árbol de regresión $\hat{f}^b(x)$
  - Promedie las muestras de bootstrap

$$\boxed{\hat{f}_{bag} = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x)} \tag{10}$$

$\rightarrow E(\hat{f}_{bag}) = E(\hat{f}^b)$

$V(\hat{f}_{bag}) = \frac{V(\hat{f})}{B}$!

$T_1 \quad T_2 \quad \ldots$

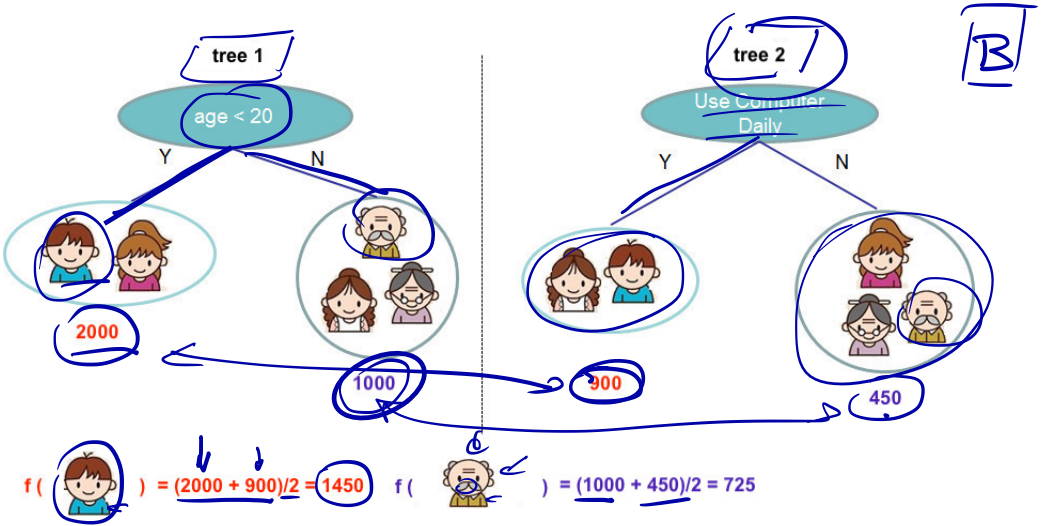
$B_1 \quad B_2$

Bagging

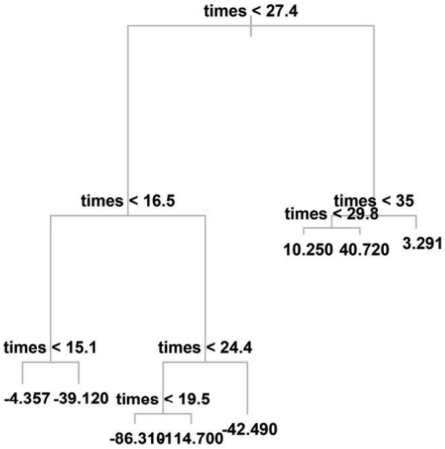
- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).
  - Para cada muestra, ajuste un árbol de regresión $\hat{f}^b(x)$
  - Promedie las muestras de bootstrap

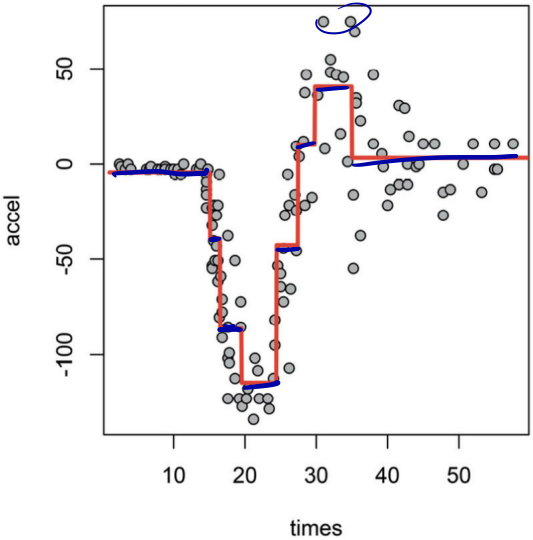
$$\hat{f}_{bag} = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x) \tag{10}$$

Bagging

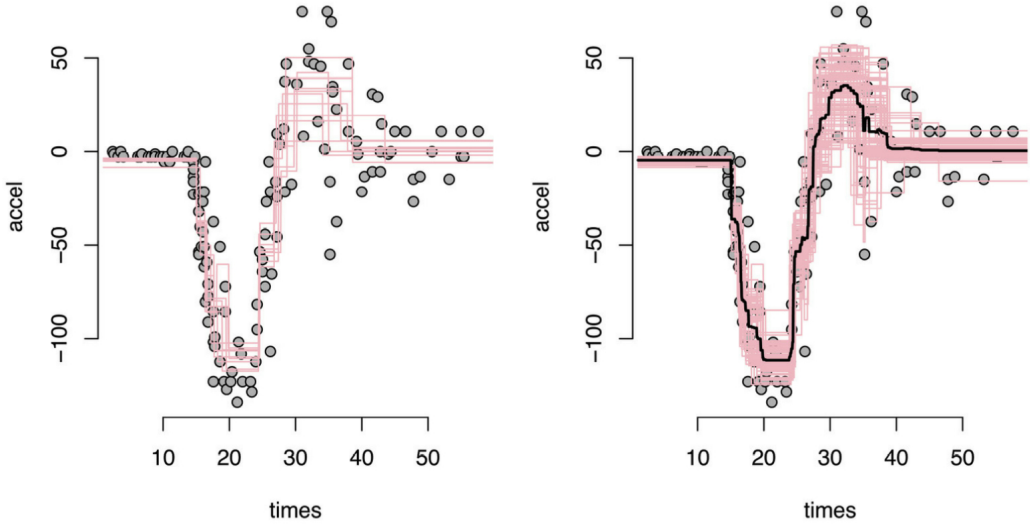

tree 1
age < 20
Y
N
2000
1000
tree 2
Use Computer Daily
Y
N
900
450
B
f ( ) = (2000 + 900)/2 = 1450
f ( ) = (1000 + 450)/2 = 725

Bagging

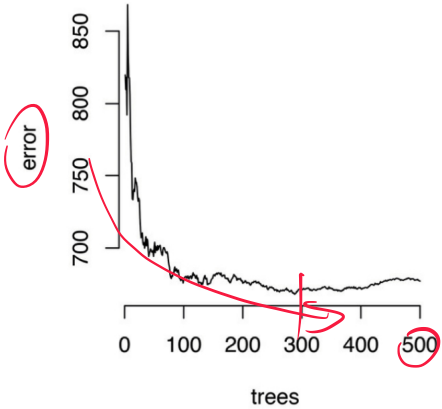

times < 27.4
times < 16.5
times < 35
times < 29.8
10.250
40.720
3.291
times < 15.1
times < 24.4
-4.357
-39.120
times < 19.5
-86.310
114.700
-42.490


accel
50
0
-50
-100
10
20
30
40
50
times

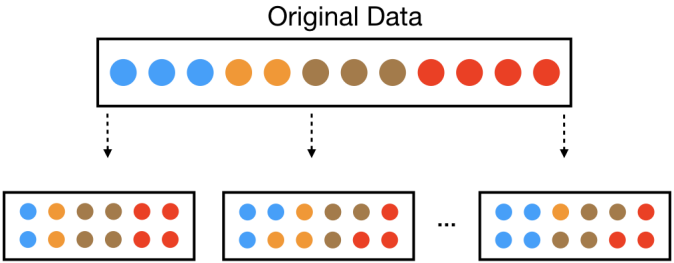
Bagging


accel
50
0
−50
−100
10
20
30
40
50
times
accel
50
0
−50
−100
10
20
30
40
50
times

Bagging



Bagging & RF



Ejemplo

photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Random Forests

- Problema con el bagging: si hay un predictor dominante en la muestra de entrenamiento, diferentes árboles son muy similares entre sí.
- Bosques (forests): reduce la correlación entre los árboles en el boostrap.
- Si hay p predictores, en cada partición use solo $m < p$ predictores, elegidos al azar.
- Bagging es forests con $m = p$ (usando todo los predictores en cada partición).
- m es un hiper-parámetro, $m = \sqrt{(p)}$ es un benchmark

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Bagging & RF

Out-of-Bag Error: Intuición

- En Bagging, entrenamos múltiples modelos sobre muestras bootstrap.
- Cada muestra contiene $\sim$63% de los datos originales. (Quiz)
- Estas observciones no usadas se llaman Out-of-Bag (OOB).

Bagging & RF

Out-of-Bag Error: Matemáticamente

Sean:

- $\{(x_i, y_i)\}_{i=1}^n$: conjunto de entrenamiento.
- $T_1, \ldots, T_B$: arboles entrenados en cada muestra bootstrap.
- $\mathcal{B}_i \subset \{1, \ldots, B\}$: índices de modelos donde x_i no fue usado (OOB).

Entonces, el error OOB es:

$$E_{OOB} = \frac{1}{n} \sum_{i=1}^{n} L\left(y_i, \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} T_b(x_i)\right)$$

- L función de pérdida (error cuadrático, $0-1$, etc.).
- Estima el **error de generalización, fuera de muestra** sin necesidad de un set de validación.

Bagging & RF

Importancia de Variables por Permutación: Intuición

W	Edu	Genero
10	Univ	M
8	Sec	F

E_{OOB}

- En modelos tipo Random Forests, queremos saber:

¿Qué tan importante es cada variable para las predicciones?

→ 10	Sec	M
→ 8	Univ	F

$E_v \uparrow$

- La idea:
  - Medimos cuánto empeora el desempeño del modelo si rompemos la relación entre una variable y el objetivo.

10	Univ	F
8	Sec	M

- ¿Cómo rompemos esa relación?
  - Permutando los valores de la variable, dejando las otras igual.
- Si la variable es importante, el error debería aumentar.

Bagging & RF

Importancia por Permutación: Definición

- Sea E_{OOB}: error del modelo usando los datos OOB.
- Para cada variable j:
  - Permutamos la columna x_j en los datos OOB.
  - Calculamos el nuevo error: $E_{\text{perm}(j)}$.
- La importancia de la variable j es:

$$\text{VI}(j) = E_{\text{perm}(j)} - E_{\text{OOB}}$$

- Si x_j es irrelevante, el error no debería cambiar.
- Si x_j es clave, el error aumentará.

Ejemplo

photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

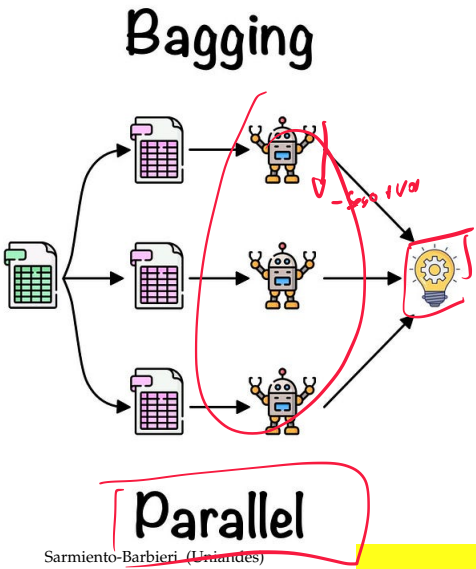
Agenda

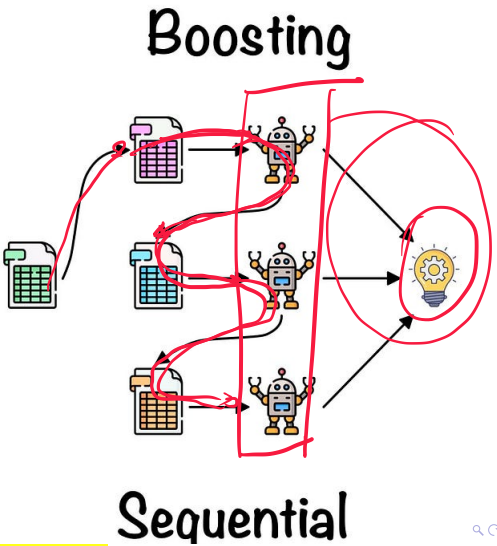
1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Boosting: Motivation

- Problema con CART: varianza alta.
- Podemos mejorar mucho el rendimiento mediante la agregación
- El boosting toma esta idea pero lo ”encara” de una manera diferente $\rightarrow$ viene de la computación
- Va a usar arboles “pequeños” y “aprende” sequencialmente

Boosting: Motivation

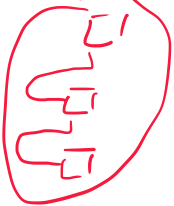




Boosting: Motivation

- Boosting es una de las ideas de aprendizaje más poderosas introducidas en los últimos veinte años.
- Originalmente fue diseñado para problemas de clasificación,
- Pero también puede extenderse a la regresión.

Boosting: Motivation



XGBoost
LightGBM

- Intuitivamente lo que hacen es "aprender" de los errores lentamente.
- Esto lo hace "ajustando" árboles pequeños.
- Esto permite mejorar lentamente aprendiendo $f(.)$ en áreas donde no funciona bien.
- OJO: a diferencia de *bagging*, la construcción de cada árbol depende en gran medida de los árboles anteriores

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

AdaBoost.M1

- El AdaBoost.M1 es una implementación para clasificación

AdaBoost.M1

1 | 1 Pobre
-1 | 0 No Pobre

Fraude | Cielo
no Fraude | no cielo

- El AdaBoost.M1 es una implementación para clasificación
- Como viene de la computación tiene un lenguaje ligeramente diferente.
- Vocabulario:
  - $y \in -1, 1$, X vector de predictores.
  - $\hat{y} = G(X)$ (clasificador)
  - $err = \frac{1}{N} \sum_i^N I(y_i \neq G(x_i))$

AdaBoost.M1

1. Comenzamos con ponderadores $w_i = 1/N$
2. Para m = 1 hasta M:
   1. Estimar $G_m(x)$ usando ponderadores w_i . Tree.
   2. Computar el error de predicción

$$err_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i} \tag{11}$$

   3. Obtener $\alpha_m = ln\left[\frac{(1-err_m)}{err_m}\right]$
   4. Actualizar los ponderadores : $w_i \leftarrow w_i c_i$

$i = 1, \ldots N.$

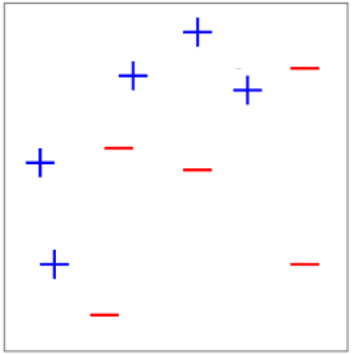
$$c_i = exp\left[\alpha_m I(yi \neq G_m(x_i))\right] \tag{12}$$

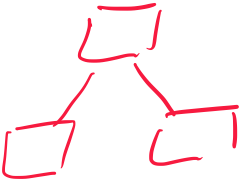
3. Resultado: $G(x) = sign\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$ $= -0{,}3 + 0{,}3 + 0{,}7 = 0{,}07$

1 -1

$-0{,}01$

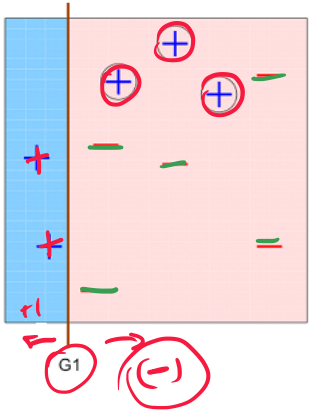
AdaBoost.M1

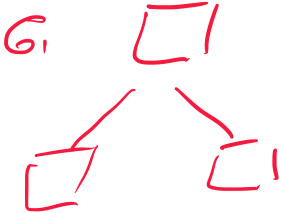




AdaBoost.M1

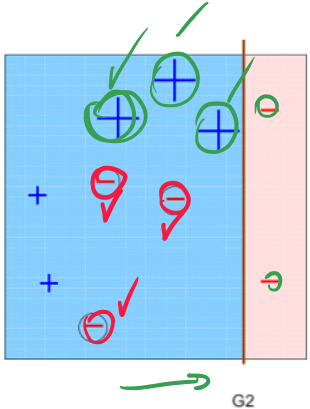
- Iteración 1

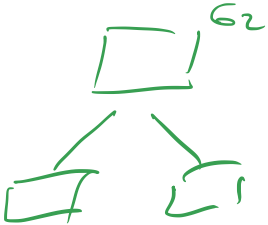




AdaBoost.M1

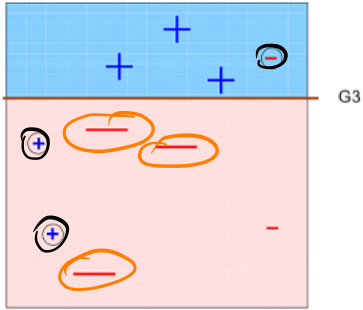
- Iteración 2





AdaBoost.M1

- Iteración 3



G_1

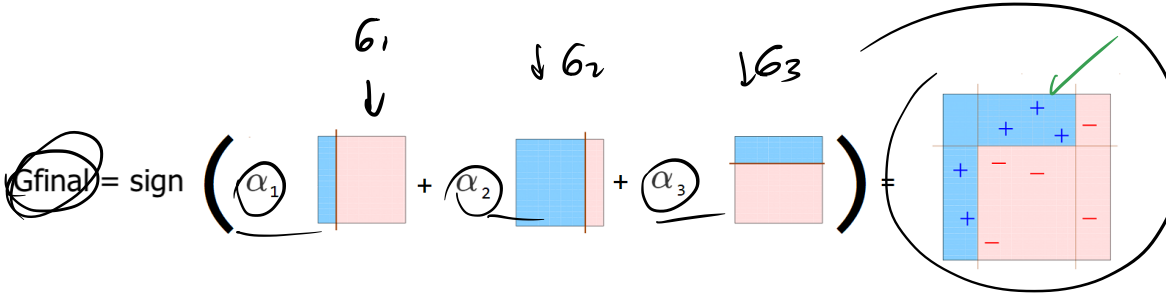
G_2

G_3

$\sum \alpha_m G_m$

AdaBoost.M1

- Resultado Final



AdaBoost.M1

1. Comenzamos con ponderadores $w_i = 1/N$
2. Para m = 1 hasta M:
   1. Estimar $G_m(x)$ usando ponderadores w_i.
   2. Computar el error de predicción

$$err_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i} \tag{13}$$

   3. Obtener $\alpha_m = ln\left[\frac{(1-err_m)}{err_m}\right]$
   4. Actualizar los ponderadores : $w_i \leftarrow w_i c_i$

$$c_i = exp\left[\alpha_m I(yi \neq G_m(x_i))\right] \tag{14}$$

3. Resultado: $G(x) = sign[\sum_{m=1}^{M} \alpha_m G_m(x)]$

AdaBoost.M1

1

$exp\left(Ln\ \frac{(1-err)}{err}\right) =$

- $c_i = exp[\alpha_m I(y_i \neq G_m(x_i))]$

$y_i \neq \hat{y}_i$

$C_i = \frac{1-err_m}{err_m} > 1$

- Si fue correctamente predicho, $\boxed{c_i = 1.}$

H.W.

Quiz

- En caso contrario, $c_i = exp(\alpha_m) = \frac{(1-err_m)}{err_m} > 1$
- En cada paso el algoritmo da mas importancia relativa a las predicciones incorrectas.
- Paso final: promedio ponderado de estos pasos

$+ \quad 1$ Pobre

$$G(x) = sign\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right] \quad = \quad - \quad 1 \quad \text{No Pobre.} \tag{15}$$

$\alpha_m \quad 1$

$+1 \quad -1$

1

-1

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

Boosting Trees

Intuition

$$\frac{\text{Err}}{\text{err}} > 1$$

- Boosting combina muchos modelos débiles (por ejemplo, árboles pequeños), entrenados **secuencialmente**, donde cada nuevo modelo se ajusta para **corregir los errores del modelo anterior**.
  - Los modelos débiles por sí solos tienen bajo desempeño (alto sesgo).
  - Boosting reduce el error al combinar varios de estos modelos.
  - Cada nuevo modelo se entrena sobre los errores (residuos) del conjunto anterior.
  - El resultado final es un modelo fuerte con mejor capacidad predictiva.

"No empezamos de cero, ajustamos lo que ya hay."

Boosting Trees

$r_i = y - \hat{y}_i$

$r_i = y_i$

stumps

Algoritmo

1. Iniciamos fijando $\hat{f}(x) = 0$ y $r_i = y_i$ para todos los i del training set
2. Para $m = 1, 2, ..., M$
   1. Ajustamos un árbol $\hat{f}^m(x) = 0$ con J bifurcaciones
   2. Actualizamos $\hat{f}(x)$

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^m(x) \tag{16}$$

   3. Actualizamos los residuales

$y_i - \lambda \hat{f}^m$

$$r_i \leftarrow r_i - \lambda \hat{f}^m(x) \tag{17}$$

3. El modelo final es

$$\hat{f}_{boost} = \sum_{m=1}^{M} \lambda \hat{f}^m(x) \tag{18}$$

Boosting Trees: Hiperparámetros

- Cuantas iteraciones (M) usar?
  - Cada iteración generalmente reduce el error de ajuste, de modo que para M lo suficientemente grande este error puede hacerse arbitrariamente pequeño (sesgo se va a cero).
  - Sin embargo, ajustar demasiado bien los datos de entrenamiento puede llevar a overfit (sobreajuste)
  - Por lo tanto, hay un número óptimo $M^\star$ que minimiza el error fuera de muestra
  - Una forma conveniente de encontrar $M^\star$ con validación cruzada

Boosting Trees: Iteraciones

- Los otros hiperparámetros a fijar son
  - λ la tasa a la que aprende, los valores típicos son 0.01 o 0.001
  - El tamaño del árbol.
    - $4 \leq J \leq 8$ funciona bien
    - $J = 2$ suele ser insuficiente
    - $J > 10$ rara vez se utiliza

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
     - XGBoost

De Boosting a Gradient Boosting

- Boosting: modelos secuenciales que corrigen errores del anterior.
- AdaBoost: ajusta pesos a las observaciones mal clasificadas.
- ¿Y si en lugar de ajustar pesos, minimizamos directamente una función de pérdida?

Del ajuste heurístico a la optimización formal.

Gradient Boosting Trees

- El objetivo es

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{19}$$

- Boosting

$$f(x_i; \{R_j, \gamma_j\}_1^J, M) = \sum_{m=1}^{M} T(x_i; \Theta_m) \tag{20}$$

- el problema es

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, \sum_{m=1}^{M} T(x_i; \Theta_m)) \right\} \tag{21}$$

Gradient Boosting Trees

- El problema

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, \sum_{m=1}^{M} T(x_i; \Theta_m)) \right\} \tag{22}$$

- Se puede aproximar por

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f_{m-1} + T(x_i; \Theta_m)) \right\} \tag{23}$$

- Ejemplo para una función de perdida cuadrática

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} (y_i - f_{m-1} - T(x_i; \Theta_m))^2 \right\} \tag{24}$$

$y_i - \hat{y}_{m-1}$

$(r_i - T(x_i, \Theta_m))^2$

Gradient Boosting Trees

- For **arbitrary** loss functions this "simplified" problem

OLS

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L\left(y_i, f_{m-1} + T(x_i; \Theta_m)\right) \right\} \tag{25}$$

- is still a tricky problem
- We can solve it by using an implementation of gradient descent

$$f_m = f_{m-1} - \epsilon \sum_{i=1}^{n} \nabla_{f_{m-1}} L(y_i, f_{m-1}(x_i)) \tag{26}$$

- How do we implement this search?

Gradient Boosting Trees

Algoritmo

1. Inicializar:

$$f_0(x) = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, \gamma)$$

2. Para $m = 1$ hasta M:
   1. Calcular los pseudo-residuos:

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

   2. Ajustar un árbol de regresión a los r_{im}, obteniendo regiones terminales R_{jm}, $j = 1, \ldots, J_m$.
   3. Para cada región R_{jm}, calcular:

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

   4. Actualizar el modelo:

$$f_m(x) = f_{m-1}(x) + \lambda \sum_{j=1}^{J_m} \gamma_{jm} \cdot I(x \in R_{jm})$$

3. El modelo final es:

$$\hat{f}(x) = f_M(x)$$

XGBoost: Motivation

- XGBoost is an implementation was specifically engineered for optimal performance and speed.
- Why talk about XGBoost?
  - Among the 29 challenge winning solutions published in Kaggle's blog during 2015, 17 solutions used XGBoost.
  - Among these solutions, eight solely used XGBoost to train the model, while most others combined XGBoost with neural nets in ensembles. (The second most popular method, deep neural nets, was used in 11 solutions)
  - The success of the system was also witnessed in 2015 Data Mining and Knowledge Discovery competition organized by ACM (KDD Cup) , where XGBoost was used by every winning team in the top-10.
  - Historically, XGBoost has performed quite well for structured, tabular data. But, if you are dealing with non-structured data such as images, neural networks are usually a better option (more on this later)

XGBoost

- Cual es la innovación? Penaliza la optimización, en cada paso la función objetivo para construir el árbol es

$$\mathcal{L} = \sum_{i=1}^{N} L(y_i, \sum_{m=1}^{M} T(x_i; \Theta_m)) + \sum_{m=1}^{M} \Omega(T(x_i; \Theta_m)) \tag{27}$$

- El segundo termino penaliza la complejidad del modelo,

$$\Omega(.) = \gamma J + \frac{1}{2}\varphi \sum_{j=1}^{J} \gamma_j^2 \tag{28}$$

$(\gamma_j - 0)^2$

lasso

ridge

XGBoost: ¿Qué mejora?

- **(1) Controla el sobreajuste**
  - Penaliza árboles muy complejos
  - Penaliza predicciones extremas

  Intuición: no solo ajusto bien, también evito "pasarme"

- **(2) Aprende de forma más inteligente**
  - No solo usa errores
  - También qué tan importantes son esos errores

  Intuición: no todos los errores pesan igual

- **(3) Es muy rápido**
  - Optimizado para computación

Boosting + regularización + eficiencia computacional

LightGBM: ¿Qué es?

Idea clave

- LightGBM = **Gradient Boosting optimizado para velocidad**

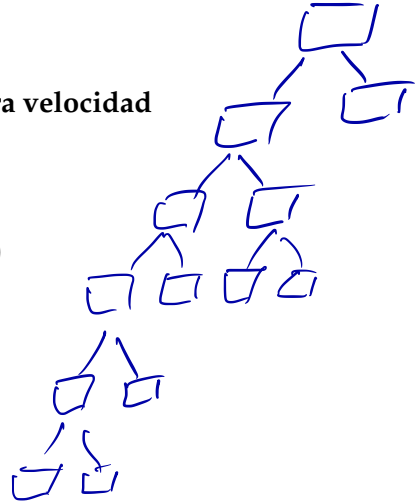
Misma estructura que boosting

$$\hat{f}(x) = \sum_{m=1}^{M} T_m(x)$$

- Cada árbol corrige errores del anterior
- Construcción secuencial

Entonces… ¿qué cambia?

- Cambia **cómo se construyen los árboles**
  - crece donde más reduce el error



Intuición

Se enfoca en las partes donde el modelo falla más

LightGBM vs XGBoost

Ambos hacen lo mismo: Boosting

La diferencia es cómo crecen los árboles

XGBoost

- Crece por niveles (level-wise)
- Árboles más balanceados
- Más estable

Intuición: crece ordenadamente

LightGBM

- Crece donde más mejora (leaf-wise)
- Árboles más desbalanceados
- Más rápido

Intuición: se enfoca en los errores grandes

Trade-off

- LightGBM: más rápido, más agresivo
- XGBoost: más estable

Misma idea, distinta estrategia

Example

photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/